

Teoremas, Proposiciones y Corolarios

Bander

Agosto 2025

Este es un resumen de los teoremas en mis palabras y con sobreexplicación seguramente para así yo lo entiendo. No pretendo reemplazar la documentación oficial, si no que es para tener una explicación para mi yo del futuro. Es una interpretación del material teórico de la materia Complejidad Computacional (1C2025).

Teoremas importantes

- Teorema de Cook Levin
- Jerarquía de Tiempos Determinísticos
- Jerarquía de Tiempos no Determinísticos
- Teorema de Ladner
- Jerarquía de Espacio
- Teorema de Savitch
- Teorema de Immerman-Szelepcsényi
- Teorema de Baker, Gill, Solovay
- Teorema de Karp-Lipton

Proposición 1: Sea $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$, sea T una función construible en tiempo y sea Γ un alfabeto. Si f es computable en tiempo $T(n)$ por una máquina $M = (\Gamma, Q, \delta)$, entonces f es computable en tiempo $O(\log|\Gamma| \cdot T(n))$ por una máquina $M' = (\Sigma, Q', \delta')$ donde $\Sigma = \{0, 1, \triangleright, \square\}$ es el alfabeto estándar.

Demostración:

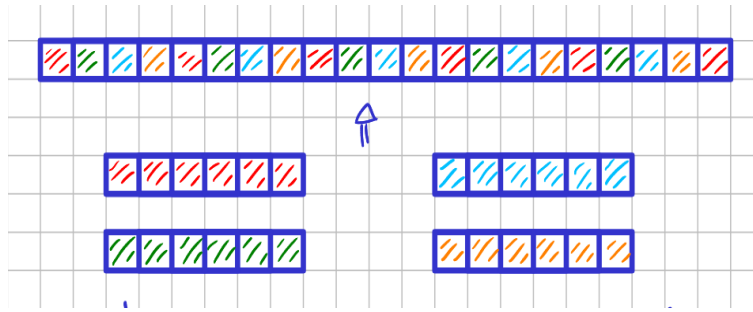
$|\Gamma|$ es la cantidad de estados que hay originalmente, para codificar cada estado de Q en Q' se usa el alfabeto Σ (binario), por lo cual conlleva $\log_2|\Gamma|$ bits por estado (por convencion usamos que $\log_2|\Gamma| = \log|\Gamma|$ bits).¹ Eso se traslada a δ' también, por lo cual, lo que antes llevaba un símbolo perteneciente a Γ ahora lleva $\log_2|\Gamma|$ símbolos de Σ (pej.: $A = 1010$).

Proposición 2: Sea $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ y sea T una función construible en tiempo. Si f es computable en tiempo $T(n)$ por una máquina estándar de $k \geq 3$ cintas (entrada, salida y $k - 2$ cintas de trabajo), entonces f es computable en tiempo $O(T(n)^2)$ por una máquina de cinta única.

Demostración:

Puedo alternar los símbolos de las k cintas en la cinta única y usar símbolos característicos para indicar dónde está la cabeza de cada cinta.

En la posición i está el caracter $\lceil \frac{i}{k} \rceil$ de la cinta $i \bmod k$.



Queda en $O(T(n)^2)$ debido a que por cada paso de δ debo primero ubicar cada cabeza para ver que accionar y después modificar cada cabeza como lo indique δ . O sea, por cada paso recorro toda la cinta 2 veces.

Proposición 3: Sea $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ y sea T una función construible en tiempo. Si f es computable en tiempo $T(n)$ por una máquina estándar, entonces hay una máquina oblivious que computa f en tiempo $O(T(n)^2)$.

Demostración:

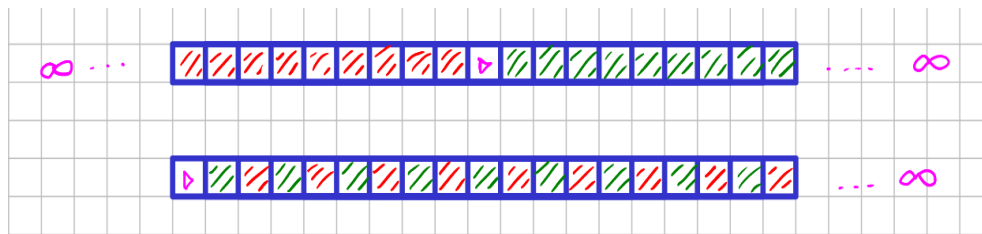
La máquina oblivious mueve un patrón fijo en cada paso (barre toda la cinta) y cambia la cinta según los cambios que se piden. Es muy similar a la [proposición 2](#).

Proposición 4:

Sea $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ y T una función construible en tiempo. Si f es computable por una máquina con cintas bi-infinitas en tiempo $T(n)$, entonces f es computable por una máquina estándar en tiempo $O(T(n))$

Demostración:

Se puede doblar la cinta bi-infinita de manera que rebota en el símbolo $\triangleright$ para ver ambos infinitos.



¹La base no importa en la notación big O para los logaritmos ya que difieren entre si por una constante multiplicativa debido a cómo se puede cambiar la base de un logaritmo ($\log_b n = \frac{\log_k n}{\log_k b}$)

Teorema 1: [Turing 1936] *halt no es computable.*

Recordatorio:

$$\text{halt}(x) = \begin{cases} 1 & \text{si la máquina con entrada } x \text{ termina } (M_x(x)) \\ 0 & \text{si no} \end{cases}$$

Demostración:

Sale por diagonalización. Tengo que:

	M_1	M_2	M_3	$\dots$
1	$M_1(1)$			
2		$M_2(2)$		
3			$M_3(3)$	
$\vdots$				$\ddots$

Defino entonces M tal que $M(x)$ termina sii $\text{halt}(x) = 0$.

$M(\langle M \rangle)$ termina $\iff \text{halt}(\langle M \rangle) = 0 \iff M(\langle M \rangle)$ no termina. **Absurdo!**

Teorema 2: *Existe una máquina U que computa la función $u(\langle i, x \rangle) = M_i(x)$. Más aún, si M_i con entrada x termina en t pasos, entonces U con entrada $\langle i, x \rangle$ termina en $c \cdot t \cdot \log(t)$ pasos, donde c depende solo de i .*

Demostración:

Pone en cada cinta de trabajo la simulación de la máquina estándar de M . O sea, en

- #1: La entrada x .
- #2: Cinta de trabajo de M .
- #3: Estado de M .

Si #3 = $[q_f]$ termina la ejecución M .

Por cada paso de M busco δ en la entrada de la máquina U .

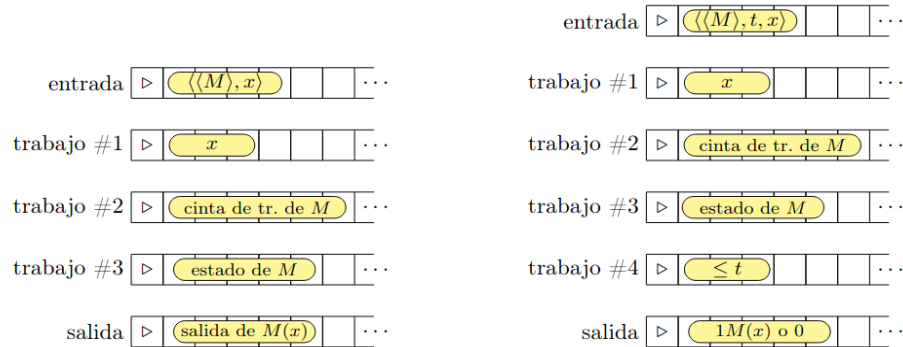


Figura 12: Izquierda: la simulación que hace U (con 3 cintas de trabajo) de M (con una única cinta de trabajo) y entrada x . Derecha: la simulación que hace $\tilde{U}$ (con 4 cintas de trabajo) de M (con una única cinta de trabajo), entrada x hasta el tiempo t .

Teorema 3: *Existe una máquina $\tilde{U}$ que computa la función $\tilde{u}(\langle i, t, x \rangle)$ en tiempo $c \cdot t \cdot \log(t)$, donde c depende solo de i .*

Demostración:

Es muy similar al **teorema 2** pero con una cinta más de trabajo para llevar registro de i (cantidad de pasos en la simulación).

Proposición 5: Si $\mathcal{L}$ es decidable en tiempo $T(n)$ por una máquina estándar de 3 cintas (entrada, trabajo y salida), entonces $\mathcal{L}$ es decidable en tiempo $O(T(n))$ por una máquina sin cinta de salida.

Teorema 4: $P \subseteq NP$

Demostración:

Sea $\mathcal{L} \in P$. $\mathcal{L}(M)$, con M una máq. det. que corre en tiempo polinomial.

Tomás el polinomio p de la definición como $p(|x|) = 0$. Defino M' det. y que corre en tiempo poly. y el certificado $c = \varepsilon$ (donde ε es la cadena vacía), entonces tengo que M' con entrada $\langle x, c \rangle$ copia el comportamiento de M con entrada x .

$$\begin{aligned} x \in \mathcal{L} &\iff M(x) = 1 \\ &\iff M'(\langle x, c \rangle) = 1 \\ &\iff \exists c. c \in \{0, 1\}^0 \text{ tal que } M'(\langle x, c \rangle) \text{ (Definición de NP)} \end{aligned}$$

Teorema 5: $NP = \bigcup_{c \in \mathbb{N}} NDTIME(n^c)$

Demostración:

$\bigcup_{c \in \mathbb{N}} NDTIME(n^c) \subseteq NP$:

Sea $\mathcal{L} \in \bigcup_{c \in \mathbb{N}} NDTIME(n^c)$ quiero ver que $\mathcal{L} \in NP$. Sea N una máq. no-det. y p un polinomio tal que N corre en tiempo $p(n)$ y $\mathcal{L}(N)$.

Existe una máq. det. M tal que M con entrada $\langle x, u \rangle$ verifica que u sea la codificación del cómputo aceptador de N a partir de x . M simula N con entrada x paso a paso, y en cada iteración i usa la primera o la segunda componente de δ según el valor de $u(i)$ para saber qué hacer.

Luego, $x \in \mathcal{L} \iff \exists u. u \in \{0, 1\}^{p(|x|)}$ tal que $M(\langle x, u \rangle) = 1$. Como M corre en tiempo polinomial (ya que es solo recorrer las decisiones en N con u), concluimos que $\mathcal{L} \in NP$.

$NP \subseteq \bigcup_{c \in \mathbb{N}} NDTIME(n^c)$:

Sea $\mathcal{L} \in NP$ quiero ver que $\mathcal{L} \in \bigcup_{c \in \mathbb{N}} NDTIME(n^c)$. Para esto uso la máq. M det. que corre en tiempo poly de la definición y la simulo con una no-det. que hace lo mismo pero inventando el certificado u por lo que te queda que corre en tiempo poly al ser que M corría en tiempo poly.

Teorema 6: Existe una máquina no-determinística NU tal que NU acepta $(\langle i, x \rangle)$ sii N_i acepta x y si N_i corre en tiempo $T(n)$ entonces $NU(\langle i, x \rangle)$ decide si N_i acepta o rechaza x en $c \cdot T(|x|)$ pasos, donde c depende solo de i .

Demostración:

Similar al teorema 2. No tiene el logaritmo porque puede adivinar no deterministamente la secuencia de elecciones que seguiría N_i para aceptar x .

Teorema 7: La relación $\leq_p$ es transitiva.

Demostración:

Sea $\mathcal{L} \leq_p \mathcal{L}'$ vía g quiero ver que $\mathcal{L} \leq_p \mathcal{L}''$.

$$x \in \mathcal{L} \iff f(x) \in \mathcal{L}' \iff g(f(x)) \in \mathcal{L}''$$

Ahora quiero ver que $g \circ f$ es computable en tiempo polinomial respecto $|x|$.

Sea M_f, M_g tal que M_f computa f en tiempo poly y M_g computa g en tiempo poly.

$M_{g \circ f} : \langle x \rangle$
 $y := M_f(x) \quad O(n^c)$ A lo sumo su salida es polinomial respecto n ($n = |x|$)
 return $M_g(y) \quad O((n^c)^d) = O(n^{cd})$ A lo sumo es poly respecto a $|y|$

Teorema 8: Si $NP\text{-hard} \cap P \neq \emptyset$, entonces $P = NP$.

Demostración:

Si $\mathcal{L} \in NP\text{-hard} \cap P$, entonces puedo tomar cualquier $\mathcal{L}' \in NP$ y reducirlo a $\mathcal{L}$, por lo cuál tengo una f computable poly tal que:

$$x \in \mathcal{L}' \iff f(x) \in \mathcal{L}$$

Entonces a la $\mathcal{L}'(M_{\mathcal{L}'})$ la declaro como:

$$\begin{aligned} M_{\mathcal{L}'} : \langle x \rangle \\ y := M_f(x) & \quad O(n^c) \\ M_{\mathcal{L}'}(y) & \quad O(n^{cd}) \end{aligned}$$

$M_{\mathcal{L}'}$ es entonces una máq. det. que corre en tiempo poly por lo que $\mathcal{L}' \in \text{P}$. O sea $\text{P} = \text{NP}$, al ser que $\mathcal{L}'$ es genérico.

Teorema 9: Si $\mathcal{L} \in \text{NP-completo}$, entonces $\mathcal{L} \in \text{P} \iff \text{P} = \text{NP}$.

Demostración:

$\mathcal{L} \in \text{P} \Rightarrow \text{P} = \text{NP}$:

Si $\mathcal{L}$ es NP-completo y a su vez $\mathcal{L} \in \text{P}$, lo único que agrega que $\mathcal{L} \in \text{NP-completo}$ es que $\mathcal{L} \in \text{NP-hard}$.² Luego, por el **teorema 8** obtengo que $\text{P} = \text{NP}$.

$\text{P} = \text{NP} \Rightarrow \mathcal{L} \in \text{P}$:

Si $\text{P} = \text{NP}$, entonces todos los problemas de NP se pueden resolver en tiempo poly, en particular también todos los NP-completos.

Proposición 6: TMSAT $\in$ NP-completo.

Recordatorio:

$$\text{TMSAT} = \{ \langle y, x, 1^n, t^t \rangle \mid \exists u \in \{0, 1\}^n \ M_y(xu) = 1 \text{ en a lo sumo } t \text{ pasos} \}$$

Demostración:

Sea $\mathcal{L} \in \text{NP}$:

$$\begin{aligned} x \in \mathcal{L} & \iff \exists u \in \{0, 1\}^{p(|x|)} : M(x, u) = 1 && \text{definición de } \mathcal{L} \text{ en NP} \\ x \in \mathcal{L} & \iff f(x) \in \text{TMSAT} && \text{definición } \mathcal{L} \leq_p \text{TMSAT} \end{aligned}$$

La f de la reducción sería:

$$f(x) = \langle \langle M \rangle, x, 1^{p(|x|)}, 1^{q(|x|+p(|x|))} \rangle \quad (\text{lo cual es computable en tiempo poly})$$

$$\begin{aligned} f(x) \in \text{TMSAT} & \iff \langle \langle M \rangle, x, 1^{p(|x|)}, 1^{q(|x|+p(|x|))} \rangle \in \text{TMSAT} \\ & \iff \exists u \in \{0, 1\}^{p(|x|)} : M(x, u) = 1 \text{ en } \leq q(|x|, p(|x|)) \text{ pasos} \\ & \iff x \in \mathcal{L} \end{aligned}$$

Proposición 7: Sea $\varphi(p_1, \dots, p_n)$ una fórmula booleana. Si v y v' son dos valuaciones tales que $v(p_i) = v'(p_i)$ para $i \in \{1, \dots, n\}$, entonces $v \models \varphi(p_1, \dots, p_n)$ sii $v' \models \varphi(p_1, \dots, p_n)$.

Demostración:

$v(p_i) = v'(p_i)$ significa que la valuación evalúa a las variables proposicionales con el mismo valor. La proposición sale inmediata.

Teorema 10: SAT, 3SAT $\in$ NP

Demostración:

$$\begin{aligned} M : \langle x, u \rangle \\ x & \text{ representa una fórmula booleana} \\ \text{if } (x = \langle y \rangle \text{ y } \varphi \text{ tiene } m \text{ variables proposicionales}) \\ \text{ret } u \upharpoonright m & \models \varphi \quad (\upharpoonright \text{ toma los primeros } m \text{ elementos de } u) \end{aligned}$$

Proposición 8: Para toda $F : \{0, 1\}^\ell \rightarrow \{0, 1\}$ existe una fórmula booleana $\varphi_F(p_1, \dots, p_\ell)$ en CNF tal que $v \models \varphi$ sii $F(v) = 1$ para todo $v \in \{0, 1\}^\ell$. Más aún, φ_F se computa en tiempo polinomial a partir de $\langle F \rangle$ y tiene tamaño $O(\ell 2^\ell)$.

Demostración:

Concateno todas las posibles combinaciones de ℓ bits ($\ell 2^\ell$) de manera que si $F(v)$ da 0, entonces $v \not\models \varphi$ y esto se hace negando la conjunción de los bits de la v en la cadena de conjunciones principal de la fórmula.

² $\mathcal{L}$ ya era NP, porque $\text{P} \subseteq \text{NP}$

Corolario 1: Para toda $F : \{0, 1\}^\ell \rightarrow \{0, 1\}^k$ existe una fórmula booleana $\varphi(p_1, \dots, p_{\ell+k})$ en CNF tal que $uv \models \varphi_F$ si y sólo si $F(u) = v$ para todo $u \in \{0, 1\}^\ell$, $v \in \{0, 1\}^k$. Más aún, φ_F se computa en tiempo polinomial a partir de $\langle F \rangle$ y tiene tamaño $O((\ell + k)2^{\ell+k})$.

Demostración:

Defino $G : \{0, 1\}^\ell \rightarrow \{0, 1\}^k$ tal que $G(u, v) = 1 \iff F(u) = v$ y aplico la **proposición 8**.

Teorema 11: (Teorema de Cook Levin) SAT $\in$ NP-hard.

Idea de la demostración: (no la explico toda porque es muy técnica)

$$\begin{aligned} x \in \mathcal{L} &\iff \exists u \in \{0, 1\}^{p(|x|)} : M(x, u) = 1 \\ &\iff \text{Existe } u \in \{0, 1\}^{p(|x|)} \text{ y existe un cómputo } C_0, \dots, C_{t(|xu|)} \text{ a partir de la entrada } xu \end{aligned}$$

Con el uso de mini-configuraciones $z_0, \dots, z_m \in \{0, 1\}^k$ con k dependiente de la máquina.

Con la entrada (xu) quiero ver si con fórmulas de tamaño polinomial a la entrada puedo simular el cómputo de la máquina M que decide a $\mathcal{L}$.

Las fórmulas son:

1. La que representa la cinta de entrada. (xu)
2. La configuración inicial de M a partir de (xu) . (q_0)
3. La evolución en un paso de z_i a z_{i+1} . (δ)
4. La configuración final de M aceptadora. $(q_f \text{ y el símbolo en la cinta de trabajo es } 1)$

Lo más difícil de ver en mi opinión es el punto 3 así que lo voy a detallar:

Para obtener $\langle z_i \rangle$ necesito usar $\langle z_{i-1} \rangle$, $\langle z_{prev(i,n)} \rangle$ y $\langle xu(e(i, n)) \rangle$ en la función F que se muestra en la definición de mini-configuraciones la cual representa la evolución en un paso del cómputo de M con entrada xu .

Luego, por el **corolario 1** obtengo una fórmula que representa esta función.

Corolario 2: SAT, 3SAT $\in$ NP-completos.

Demostración:

Sale del **teorema 10 y 11**.

Proposición 9: INDSET $\in$ NP-completo.

Demostración:

Ver que INDSET $\in$ NP es medianamente fácil, ahora ver que INDSET $\in$ NP-hard tenés que hacerlo por definición (nunca) o por reducción polinomial de un NP-hard que ya sepamos.

Veamos que 3SAT $\leq_p$ INDSET:

$$\varphi = (\ell_{11} \vee \ell_{12} \vee \ell_{13}) \wedge \dots \wedge (\ell_{m1} \vee \ell_{m2} \vee \ell_{m3})$$

A lo sumo tengo $3m$ vértices (si todos los literales son distintos), uno por cada variable, m = cantidad de cláusulas.

Defino una arista entre z y z' si están en la misma cláusula de la fórmula o la variable ligada al nodo z es la negación de la variable ligada al nodo z' .

Esto sería la f de que:

$$x \in 3SAT \iff f(x) \in \text{INDSET} \quad (\text{La demo de que esto funciona queda en tu imaginación})$$

Proposición 10: CAMHAM $\in$ NP-completo.

Teorema 12: TSP $\in$ NP-completo.

Proposición 11: KNAPSACK $\in$ NP-completo

Teorema 13: Si $P = NP$ entonces $\text{ExpTime} = \text{NExpTime}$

Demostración:

$\text{ExpTime} \subseteq \text{NExpTime}$:

Esto ya se cumple sin que $P = NP$.

$\text{NExpTime} \subseteq \text{ExpTime}$:

Sea $\mathcal{L} \in \text{NExpTime}$ y $\mathcal{L}(N)$ tal que N es no-det. y corre en $c2^{n^c}$.

Considero $\mathcal{L}_{pad} : \{\langle x, 1^{2^{|x|^c}} \rangle : x \in \mathcal{L}\}$ ($\mathcal{L}$ con padding)

Veamos que $\mathcal{L}_{pad} \in NP$:

Como la entrada de $\mathcal{L}_{pad}$ es una cadena exponencial con respecto a $|x|$, puedo correr a N con x por $2^{|x|^c}$ pasos porque es poly con respecto a $|\langle x, 1^{2^{|x|^c}} \rangle|$.

Teorema 14: (Jerarquía de Tiempos Determinísticos) Si f, g son construibles en tiempo y cumplen que $f(n) \cdot \log(f(n)) = o(g(n))$ entonces $\text{DTime}(f(n)) \subsetneq \text{DTime}(g(n))$.

Demostración:

A ver, esto va a ser lento y detallado porque me cuesta entenderla.

La idea principal es demostrar por diagonalización que hay lenguajes que estan en $\text{DTime}(g(n))$ pero no en $\text{DTime}(f(n))$.

Declaro D máq. det. tal que:

$D : \langle x \rangle$

Calcula $n = |x|$ y $t = g(n)$ $O(g(n) + n) = O(g(n))$

Simula $U(\langle x, x \rangle)$ por t pasos $O(g(n))$

if($U(\langle x, x \rangle)$ no terminó en a lo sumo t pasos): $O(1)$

ret 0

else:

ret $\neg(U(\langle x, x \rangle))$

$\mathcal{L} \in \text{DTime}(g(n))$ es inmediato.

Ahora vamos a ver por absurdo la prueba:

Supongo que $\mathcal{L}(D) \in \text{DTime}(f(n))$. Sea $\mathcal{L}(M) = \mathcal{L}(D)$, donde M termina en a lo sumo $O(f(|x|))$ pasos (c.f. $|x|$) acá $|x| \geq n_0$ que es el punto donde siempre la función f se vuelve mayor

Por el teorema 2 sé que $U(\langle x, x \rangle)$ simula $M_x(x)$ en $c' \cdot c \cdot f(|x|) \cdot \log(c \cdot f(|x|))$ pasos.³

Luego, existen d y $n_1 \geq n_0$ tal que $\forall n \geq n_1$:

$$c' \cdot c \cdot f(|x|) \cdot \log(c \cdot f(|x|)) \leq d \cdot f(n) \cdot \log(f(n)) \leq g(n)$$

Tomo x tal que $|x| \geq n_1$ y $M_x = M$. La existencia de este x es garantizada porque se pueden hacer infinitas máquinas que hagan lo mismo.

Ahora bien, $U(\langle x, x \rangle)$ termina en $g(|x|)$ pasos. Luego, $D(x) = \neg(U(\langle x, x \rangle)) = \neg(M_x(x)) = \neg(M(x))$ Absurdo! Si habíamos dicho que $\mathcal{L}(D) = \mathcal{L}(M)$

Ok, tiene sentido, te armás una máquina que corra en $g(|x|)$ pasos, forzas a una máquina a identificar el mismo lenguaje pero en $f(|x|)$ pasos y como siempre termina, siempre pasa que da una contradicción con el lenguaje que se suponía que decidía.

¿Qué pasa si le meto D_x ? ¿No me da una contradicción también?

$D(\langle D_x \rangle) = U(\langle D_x, \langle D_x \rangle \rangle)$ por t pasos = $g(|x|)$ pasos ... Opa? Esto no termina, o sea, si que termina, retorna 0 porque no termina el llamado recursivo que se llama a $U(\langle D_x, \langle D_x \rangle \rangle) = D(\langle D_x \rangle)$.

Teorema 15: (Jerarquía de Tiempos no Determinísticos) Si f, g son construibles en tiempo, no decrecientes y cumplen que $f(n+1) = o(g(n))$ entonces $\text{NDTime}(f(n)) \subsetneq \text{NDTime}(g(n))$.

Demostración:

Sea $(N)_{i \in \mathbb{N}}$ una enumeración de todas las máquinas no-det. = $\{N_1, N_2, \dots\}$.

³el teorema 2 dice que $U(\langle x, x \rangle)$ corre en $k \cdot z \cdot \log(z)$ donde z es el tiempo de cómputo de x en la máquina M_x

Definimos N con la entrada x así:

x debe ser de la forma $1^i 0y$, si no **rechaza**

Si $|y| < g(i+1)$:

Simular no-det $N_i(x0)$ y $N_i(x1)$ por $g(|x|)$ pasos

Ambas deben terminar y aceptar para **acepta**, si no, **rechaza**

Si $|y| = g(i+1)$:

Simular no-det $N_i(1^i 0)$ siguiendo el cómputo codificado por y

Si termina y rechaza, entonces **acepta**, si no, **rechaza**

Si $|y| > g(i+1)$:

Rechaza

Bueno, esta máquina lo que hace básicamente es una recursión hasta que $|y| = g(i+1)$ (si $|y| \leq g(i+1)$), tomar en cuenta que $N_i(x0) = NU(i, x0)$, pero sabemos por el **teorema 6** que toma el mismo tiempo hacer uno o lo otro.

Veamoslo por el absurdo, sea $\mathcal{L}(n) \in \text{NDTime}(f(n))$, hay una máquina no-det N' tal que $\mathcal{L}(N) = \mathcal{L}(N')$ y N' con entrada x suficientemente larga termina en $c \cdot f(n)$ pasos.

Luego, tomamos i lo suficientemente grande para que:

1. $N' = N$

2. Para todo $y \in \{0, 1\}^*$, si $|x| > i$ entonces $N_i(x0)$ y $N_i(x1)$ terminan en tiempo no-det $g(|x|)$

Ahora tomando que $\mathcal{L}(N) = \mathcal{L}(N_i) = \mathcal{L}(N')$, obtenemos que con la entrada $1^i 0$:

$$1^i 0 \in \mathcal{L}(N) \iff \forall y \in \{0, 1\}^* 1^i 0y \in \mathcal{L}(N_i)$$

$$\iff \forall y \in \{0, 1\}^* 1^i 0y \in \mathcal{L}(N_i)$$

...

$$\iff \forall y \in \{0, 1\}^{g(i+1)} 1^i 0y \in \mathcal{L}(N_i)$$

$$\iff N_i(1^i 0) \text{ rechaza siguiendo todos los cómputos } y \text{ tal que } |y| = g(i+1)$$

$$\iff 1^i 0 \notin \mathcal{L}(N_i) = \mathcal{L}(N) \quad \text{Absurdo!}$$

Lo que podemos ver aca es que como se que N_i termina en $f(n)$ pasos y cuando uso N para ver que $1^i \in \mathcal{L}(N)$ entra en la primera condicion donde $|y| < g(i+1)$ y se pone a simular $N_i(x0)$ y $N_i(x1)$ lo cual tarda $f(|x|+1)$ lo cual sabemos que es menor estricto para algun x lo suficientemente grande a $g(|x|)$.

O sea que estos cómputos terminan y como identifican el mismo lenguaje que N estas aceptan sii

$\forall y \in \{0, 1\}^{g(i+1)} 1^i 0y \in \mathcal{L}(N_i)$, o sea que, (por la segunda condición que es un caso base $|y| = g(i+1)$) sé que $N_i(1^i 0)$ rechaza siguiendo todos los cómputos y tal que $|y| = g(i+1)$ por lo cual $1^i 0 \notin \mathcal{L}(N_i) = \mathcal{L}(N)$.

Teorema 16: (Teorema de Ladner) Si $P \neq NP$ entonces existe $\mathcal{L}$ tal que $\mathcal{L} \in NP$, $\mathcal{L} \notin P$, y $\mathcal{L} \notin NP$ -completo.

Demostración:

Quiero ver que si $P \neq NP \Rightarrow \exists \mathcal{L} : \mathcal{L} \in NP$, $\mathcal{L} \notin P$, y $\mathcal{L} \notin NP$ -completo.

¿Cómo hago esto? Me creo un $\mathcal{L}$ y voy viendo que asumiendo el antecedente $P \neq NP$, llego a obtener que se cumple el consecuente si lo hice bien al $\mathcal{L}$.

Definimos el problema SAT_H relativo a la funcion $H : \mathbb{N} \rightarrow \mathbb{N}$ tal que:

$$\text{SAT}_H = \{\psi 01^{n^{H(n)}} : \psi \in \text{SAT}, n = |\psi|\}$$

Es como un **SAT** con padding, o sea, no agrega nada a ψ

Ahora, si $H(n) = n \Rightarrow \text{SAT}_h \in P$: Esto pasa porque la entrada sería $\psi 01^{n^n}$, por lo cual, resolver que ψ es **SAT** en una M det tomaría 2^n , lo cual es polinomial respecto a la entrada (porque $|2^n| \leq |1^{n^n}|$ para toda ψ salvo finitos casos).

Si $H(n) = \text{constante} \Rightarrow \text{SAT}_H \in \text{ExpTime}$: El tener $\psi 01^{n^c}$ me deja la entrada con $|\psi 01^{n^c}| = n^{c+1}$ y esto no es gran cambio para el **SAT** original, y sigue siendo **NP-completo** SAT_H .

Necesito que $H(n)$ tienda a infinito pero que crezca lentamente para llegar a un **NP-intermedio**.

Se define H tal que:

$$H(x) = \begin{cases} \star \min i < \log(\log(n)) \text{ tal que } \forall x . x \in \{0, 1\}^{\leq \log(n)} M_i(x) = \mathcal{X}\text{SAT}_H(x) \text{ en a lo sumo } i \cdot |x|^i \text{ pasos} & \text{si existe tal } i \\ \log(\log(n)) & \text{si no} \end{cases}$$

★ No entendi que hace acá, veamoslo más tranqui:

Buscamos si alguna máquina M_i decide SAT_H sobre todas las entradas chicas ($\leq \log(n)$) en a lo sumo $i|x|^i$ pasos.

El i debe ser el mínimo menor a $\log(\log(n))$

Si no existe, ponemos $H(n) = \log(\log(n))$ para que crezca un poco.

SAT_H y H quedan definidas por recursión mutua (H depende de SAT_H y viceversa)

- $H(n)$ usa $\text{SAT}_H(x)$ para palabras x tal que $|x| \leq \log(n)$
- $\text{SAT}_H(x)$ usa $H(n)$ para palabras x tal que $|x| > n$

donde $x = \psi 01^{n^{H(n)}}$ y $n = |\psi|$

Proposición 12: $\text{SAT}_H \in \text{P} \Rightarrow \exists c \forall n H(n) \leq c$.

Proposición 13: $\exists c \exists^\infty n H(n) \leq c \Rightarrow \text{SAT}_H \in \text{P}$.

Proposición 14: Existe k tal que para toda fórmula booleana ψ , $|\psi| > k \Rightarrow |F_\psi| < |\psi|$.

Proposición 15: $\text{DTime}(S(n)) \subseteq \text{Space}(S(n))$.

Proposición 16: $\text{Space}(S(n)) \subseteq \text{NSpace}(S(n))$.

Proposición 17: Sea M una máquina (determinística o no determinística) que usa espacio $S(n)$ tal que $S(n) \geq \log(n)$. Existe una constante c tal que para todo $x \in \{0,1\}^*$, cada vértice de $G_{M,x}$ se puede describir usando $c \cdot S(|x|)$ bits (c depende de M). Luego, $G_{M,x}$ tiene $2^{c \cdot S(|x|)}$ nodos.

Proposición 18: Si S es construible en espacio, $\text{NSpace}(S(n)) \subseteq \text{DTime}(2^{O(S(n))})$.

Proposición 19: $\text{NP} \subseteq \text{PSpace}$.

Teorema 17: (Jerarquía de Espacio) Si f, g son construibles en espacio y cumplen que $f(n) = o(g(n))$ entonces:

$$\text{Space}(f(n)) \subsetneq \text{Space}(g(n)).$$

Teorema 18: $\text{TQBF} \in \text{PSpace}$.

Proposición 20: En tiempo polinomial se puede transformar cualquier QBF generalizada $\psi(y_1, \dots, y_m)$ en una QBF $\psi'(y_1, \dots, y_m)$ posiblemente con variables libres que es equivalente, es decir, tal que para todo $v \in \{0,1\}^m$

$$v \models \psi(y_1, \dots, y_m) \iff v \models \psi'(y_1, \dots, y_m).$$

Proposición 21: $\text{CHECKQBF} \leq_p \text{TQBF}$.

Proposición 22: Sea M una máquina (determinística o no-determinística) que usa espacio $S(n) \geq \log(n)$ y sea c una constante tal que para todo $x \in \{0,1\}^*$ cualquier configuración C de un cómputo de M con entrada x se representa con $|\langle C \rangle| = c \cdot S(|x|)$ bits. Dado $x \in \{0,1\}^*$, podemos computar en tiempo polinomial en $S(n)$ una fórmula $\varphi_{M,x}(\bar{s}, \bar{t})$ en CNF con variables libres $\bar{s} = s_1, \dots, s_{c \cdot S(|x|)}$, $\bar{t} = t_1, \dots, t_{c \cdot S(|x|)}$ tal que $\langle C \rangle \langle C' \rangle \models \varphi_{M,x}(\bar{s}, \bar{t})$ sii hay una flecha de C a C' en $G_{M,x}$.

Demostración:

Similar a Cook Levin lo que hacemos (ahora con configuraciones completas en vez de mini-configuraciones) es expresar cuando una configuración completa C codificada con las variables $\bar{t}$, es la evolución en un paso de otra configuración C' codificada con variables $\bar{s}$.

Para saber lo anterior alcanza con mirar qué pasa en la codificación de algunos bits de la cinta para ver donde está la cabeza de las cintas y como ya tengo la δ puedo simplemente comparar cuales son las acciones necesarias para llegar a como estan las cosas en C' y ver si es posible con alguna transición de la función δ .

Teorema 19: $\text{TQBF} \in \text{PSpace-hard}$.

Demostración:

Sea $\mathcal{L} \in \text{PSpace}$ quiero ver que :

$$\mathcal{L} \leq_p \text{TQBF}$$

$$x \in \mathcal{L} \iff f(x) \in \text{TQBF}$$

Para esto uso un lenguaje intermedio tal que:

$$\mathcal{L} \leq_p \text{CHECKQBF} \leq_p \text{TQBF}$$

Para demostrar esto usamos la **proposición 22** para formar $\psi_{c\dot{S}(n)}$ tal que $\langle C \rangle \langle C' \rangle \models \psi_i(\bar{s}, \bar{t})$ sii existe un camino de longitud a lo sumo 2^i en $G_{M,x}$ entre C y C' .

A esta ψ la vamos a definir recursivamente, el caso base ya lo probamos en la anterior proposición y el caso recursivo es una especie de middle search así no se vuelve exponencial la búsqueda.

$$\psi_i(\bar{s}, \bar{t}) = \exists \bar{r} \forall \bar{u}, \bar{v} ((\bar{u} = \bar{s} \wedge \bar{v} = \bar{r}) \vee (\bar{u} = \bar{r} \wedge \bar{v} = \bar{t})) \rightarrow \psi_{i-1}(\bar{u}, \bar{v})$$

Teorema 20: (Teorema de Savitch) $\text{NSpace}(S(n)) \subseteq \text{Space}(S(n)^2)$, de modo que $\text{PSpace} = \text{NPSpace}$.

Demostración:

Sea $\mathcal{L} \in \text{NSpace}(S(n))$ y sea N una máquina no-det tal que $\mathcal{L}(N)$, sabemos que:

$$x \in \mathcal{L} \text{ sii hay un camino de longitud a lo sumo } 2^{c \cdot S(|x|)} \text{ en } G_{N,x} \text{ desde } C_0 \text{ a } C_f$$

Definimos $\text{Reach}(C, C', i) \in \{0, 1\}$, la cual devuelve 1 si hay un camino de longitud a lo sumo 2^i en $G_{N,x}$ desde C hasta C' de la siguiente manera:

Si $i = 0$:

ret $(C \rightarrow C' \in E_G)$

Para cada posible configuración C'' de $G_{N,x}$:

$j = \text{Reach}(C, C'', i-1)$

$k = \text{Reach}(C'', C', i-1)$

Si $j = k = 1$:

ret(1)

ret(0)

Tenemos entonces que:

$$x \in \mathcal{L} \iff \text{Reach}(C_0, C_f, c \cdot S(n)) = 1$$

Luego, Reach reusa el espacio para calcular j y k , entonces $\text{Reach}(C, C', i)$ usa espacio $O(i \cdot S(n))$.

Por último, calculamos $\text{Reach}(C_0, C_f, c \cdot S(n))$ en $O(S(n)^2)$.

Proposición 23: $\text{PATH} \in \text{NL}$

Proposición 24: Si $\mathcal{L} \notin \{0, 1\}^*, \emptyset$ entonces $\mathcal{L}$ es NL-hard con respecto a $\leq_p$.

Proposición 25: Sean f, g computables implícitamente en L . Entonces $g \circ f$ es computable implícitamente en L .

Teorema 21: $\text{PATH} \in \text{NL-completo}$ y por lo tanto $\overline{\text{PATH}} \in \text{coNL-completo}$.

Teorema 22: $\mathcal{L} \in \text{NL}$ sii existe un polinomio $p: \mathbb{N} \rightarrow \mathbb{N}$ y una máquina determinística M con una cinta de entrada adicional de lectura de una única vez (lee y pasa a la siguiente celda a la derecha pero no puede volver atrás) tal que:

1. Para todo $x \in 0, 1^*$, $x \in \mathcal{L}$ sii existe $u \in 0, 1^{p(|x|)}$ tal que $M(x, u) = 1$; aquí $M(x, u)$ denota la salida de M cuando la cinta de entrada tiene x y la cinta adicional de lectura de una única vez tiene u
2. M usa espacio $O(\log(n))$; en este modelo de máquina con cinta de entrada adicional, la cinta de entrada y la cinta adicional no cuentan en el espacio (solo cuentan sus cintas de trabajo y salida).

Teorema 23: (Teorema de Immerman-Szelepcsényi) $\overline{\text{PATH}} \in \text{NL}$.

Demostración:

Nuestro objetivo es ver que podemos certificar que $\langle G, s, t \rangle \notin \text{PATH}$. Para esto vamos a usar:

$$A_i = \{v \in V : v \text{ es alcanzable desde } s \text{ en a lo sumo } i \text{ pasos}\}$$

Luego,

$$\langle G, s, t \rangle \notin \text{PATH si } t \notin A_n$$

Para esta demostración vamos a crear el certificado final constructivamente y en cada paso demostrar que se puede verificar con una máquina que usa espacio $O(\log(n))$ y que el tamaño de la parte del certificado que estamos haciendo es polinomial al tamaño de la entrada.

1. **Certificado de que $v \in A_i$:** $Z_{v \in A_i}$ = Lista de nodos de s a v donde cada elemento de la lista tiene una arista con el elemento que tiene al lado.
2. **Certificado de que $v \notin A_i$ conociendo $|A_i|$:** $Z_{v \notin A_i}^{|A_i|}$ = Lista de tuplas de tamaño $|A_i|$, con el primer elemento de la tupla siendo un nodo y el segundo el certificado que dice que pertenece a A_i , esta lista contiene todos los elementos del conjunto A_i . En este certificado veo que mi v no esté en la lista y que los certificados de las tuplas den 1.
3. **Certificado de que $v \notin A_i$ conociendo $|A_{i-1}|$:** $Z_{v \notin A_i}^{|A_{i-1}|}$ = Similar al anterior pero ahora es la lista de los elementos de A_{i-1} y lo que veo con este certificado es que los elementos de esta lista en su siguiente paso no puedan ir a mi v .
4. **Certificado de que $|A_i| = a$ conociendo $|A_{i-1}|$:** $Z_{|A_i|=a}^{|A_{i-1}|}$ = Es una lista de tuplas con todos los nodos en las primeras posiciones en orden y un certificado de que ese nodo pertenece a A_i o que no pertenece y ve la cantidad que pertenecen.
5. **Certificado final de que $t \notin A_n$:** Z = Es una lista de certificados donde los primeros $n - 1$ elementos de la lista son certificados para obtener $|A_n|$ constructivamente con el certificado 4 y el último es el de certificar que $t \notin A_n$ con $|A_n|$, o sea $Z_{t \notin A_n}^{|A_n|}$.

Corolario 4: NL = coNL.

Teorema 24: Si $S(n) \geq \log(n)$ es construible en espacio entonces $\text{NSpace}(S(n)) = \text{coNSpace}(S(n))$.

Proposición 26: $\text{MAXINDSET} \in \Sigma_2^P$.

Proposición 27: La jerarquía polinomial tiene las siguientes propiedades:

- $\Sigma_1^P = \text{NP}$
- $\Sigma_i^P \subseteq \Sigma_{i+1}^P$
- $\Sigma_i^P \subseteq \Pi_{i+1}^P$
- $\text{PH} = \bigcup_{i \geq 0} \Sigma_i^P$
- $\Sigma_i^P \subseteq \Sigma_{i+1}^P$
- $\Pi_1^P = \text{coNP}$
- $\Pi_i^P \subseteq \Pi_{i+1}^P$

La pregunta $\text{P} \stackrel{?}{=} \text{NP}$ se puede generalizar a $\Sigma_i^P \stackrel{?}{=} \Sigma_{i+1}^P$; ninguna se conoce. Decimos que la jerarquía polinomial colapsa si $\text{P} = \text{PH}$.

Proposición 28: Si $\text{P} = \text{NP}$ entonces $\text{PH} = \text{P}$.

Proposición 29: Las clases Σ_i^P y Π_i^P están cerradas hacia abajo por $\leq_p$. Es decir, para $\mathcal{L}' \leq_p \mathcal{L}$ tenemos que si $\mathcal{L} \in \Sigma_i^P$ entonces $\mathcal{L}' \in \Sigma_i^P$ y si $\mathcal{L} \in \Pi_i^P$ entonces $\mathcal{L}' \in \Pi_i^P$.

Proposición 30: Si existe $\mathcal{L} \in \text{PH}$ -completo entonces existe i tal que $\text{PH} = \Sigma_i^P$.

Corolario 5: Si la jerarquía polinomial no colapsa en ningún nivel entonces $\text{PH} \neq \text{PSPACE}$.

Proposición 31: Para todo $i > 0$, $\Sigma_i \text{SAT} \in \Sigma_i^P$.

Proposición 32: Para todo $i > 0$, $\Sigma_i \text{SAT} \in \Sigma_i^P$ -hard.

Demostración:

Tomando en consideración que ya te sabés **Cook Levin** de taquito, lo importante para saber de esta demostración es que:

$$\begin{aligned} M(xu) = 1 &\iff \tilde{u} \models \exists \bar{e}, \bar{q}_0, \dots, \bar{q}_m : \psi_1 \wedge \psi_2 \wedge \psi_3 \wedge \psi_4 \\ &\iff \tilde{u} \models \forall \bar{e}, \bar{q}_0, \dots, \bar{q}_m : (\psi_1 \wedge \psi_2 \wedge \psi_3) \rightarrow \psi_4 \end{aligned}$$

Esto es una equivalencia de la lógica.

Luego, voy a ver para $\Sigma_i\text{SAT}$ que para ambos casos de la última $Q \in \{\forall, \exists\}$ se cumple simplemente usando lo dicho en **Cook Levin** y la equivalencia lógica que mostramos antes.

Corolario 6: Para todo $i > 0$, $\Sigma_i\text{SAT} \in \Sigma_i^P$ -completo.

Proposición 33: Para todo $i > 0$, $\Pi_i\text{SAT} \in \Pi_i^P$ -completo.

Proposición 34: Si $\mathcal{X} \in P$, entonces $P = P^{\mathcal{X}}$.

Proposición 35: $\text{ExpTime} \subseteq P^{\text{EXPCOM}}$.

Proposición 36: $\text{NP}^{\text{EXPCOM}} \subseteq \text{ExpTime}$.

Corolario 7: $P^{\text{EXPCOM}} = \text{NP}^{\text{EXPCOM}}$.

Teorema 25: (Teorema de Baker, Gill, Solovay) Existen oráculos $\mathcal{A}$ y $\mathcal{B}$ tal que $P^{\mathcal{A}} = \text{NP}^{\mathcal{A}}$ y $P^{\mathcal{B}} \neq \text{NP}^{\mathcal{B}}$.

Teorema 26: Para $i \geq 1$, $\Sigma_{i+1}^P = \text{NP}^{\Sigma_i\text{SAT}}$.

Teorema 27: $P \subseteq P_{/\text{poly}}$.

Demostración:

Sea $\mathcal{L} \in P$ tal que $\mathcal{L}(M)$, con M una máquina determinística que corre en tiempo polinomial $t(n)$, podemos suponer que es oblivious, sin cinta de salida y con una única cinta de trabajo.

Luego, $x \in \mathcal{L}$ sii existe una secuencia de mini-configuraciones de M con entrada x y representamos x_i con la cadena ets por lo que el tamaño de la codificación de cada z_i es constante.

Por último a C_n lo construimos tal que $M(x) = C_n$ para todo $x \in \{0, 1\}^*$ por la definición de mini-configuraciones me hago una $F_n(e, z_i, z_{\text{prev}(i,n)}) = z_{i+1}$ la cual puedo representar con un circuito de tamaño constante.

Como M es oblivious el circuito C_n cumple que $|C_n| = O(n + t(n))$.

Teorema 28: $P \not\subseteq P_{/\text{poly}}$.

Proposición 37: Sea $(C_n)_{n \in \mathbb{N}}$ una familia de circuitos de tamaño $S(n)$ tal que⁴ $C_n(\varphi) = \mathcal{X}\text{SAT}(\varphi)$ para toda fórmula booleana $\varphi = \varphi(x_1, \dots, x_n)$ con $|\varphi| = n$. Entonces existe una familia de circuitos $(C'_n)_{n \in \mathbb{N}}$ de tamaño polinomial en $S(n)$ tal que para todo n : C'_n tiene n salidas y para toda fórmula booleana $\varphi = \varphi(x_1, \dots, x_n)$ con $|\varphi| = n$, tenemos $\varphi(C'_n(\varphi)) = \mathcal{X}\text{SAT}(\varphi)$.

Demostración:

Teniendo $C_n(\varphi) = \mathcal{X}\text{SAT}(\varphi)$ puedo usarlo para ir adivinando cada elemento de una entrada que satisface a φ iterativamente.

Teorema 29: (Teorema de Karp-Lipton) Si $\text{NP} \subseteq P_{/\text{poly}}$, entonces $\text{PH} = \Sigma_2^P$.

Demostración:

Alcanza con probar que con esa premisa puedo llegar a que $\Pi_2^P = \Sigma_2^P$, o equivalentemente probar que $\Pi_2\text{SAT} \in \Sigma_2^P$. Por la premisa sé que:

$$\varphi \in \text{SAT} \iff \exists \bar{v} \in \{0, 1\}^n \models \varphi(\bar{v}) \iff C_n(\varphi) = 1$$

Luego, tenemos que por $\Pi_2\text{SAT}$:

$$\forall \bar{u} \in \{0, 1\}^n (\exists \bar{v} \in \{0, 1\}^n \models \varphi_{\bar{u}}(\bar{v}) \iff C_n(\varphi_{\bar{u}}(\bar{y})) = 1)$$

Luego,

$$\begin{aligned} \forall \bar{u} \exists \bar{v} \varphi(\bar{u}, \bar{v}) \in \Pi_2\text{SAT} \quad & \text{sii } \forall \bar{u} \in \{0, 1\}^n \exists \bar{v} \in \{0, 1\}^n \models \varphi(\bar{u}, \bar{v}) \\ & \text{sii } \forall \bar{u} \in \{0, 1\}^n C_n(\varphi_{\bar{u}}(\bar{y})) = 1 \\ & \text{sii } \forall \bar{u} \in \{0, 1\}^n \models \varphi_{\bar{u}}(C'_n(\varphi_{\bar{u}}(\bar{y}))) \\ & \text{sii } \exists c \in \{0, 1\}^{q(n)} \forall \bar{u} \in \{0, 1\}^n \models \varphi_{\bar{u}}(R_c(\varphi_{\bar{u}})) \end{aligned}$$

⁴Notamos $C_n(\varphi)$ en ve de $C_n(\langle \varphi \rangle)$ y lo mismo para C'_n

Proposición 38: $\text{minSize}(f) = O(n2^n)$ para toda $f : \{0, 1\}^n \rightarrow \{0, 1\}$.

Teorema 30: Para todo $n > 1$, existe $f : \{0, 1\}^n \rightarrow \{0, 1\}$ tal que $\text{minSize}(f) > 2^n/4n$.

Proposición 39: $\text{PARITY} \in \text{NC}_{\text{nu}}^1$.

Proposición 40: $\text{PARITY} \notin \text{AC}_{\text{nu}}^0$.

Proposición 41: $\text{SUMA} \in \text{AC}_{\text{nu}}^0$.

Proposición 42: $\text{NC}_{\text{nu}}^d \subseteq \text{AC}_{\text{nu}}^d \subseteq \text{NC}_{\text{nu}}^{d+1}$. Por lo tanto, $\text{AC}_{\text{nu}} = \text{NC}_{\text{nu}}$.

Teorema 31: $\mathcal{L}$ es decidable por una familia de circuitos P-uniforme sii $\mathcal{L} \in \text{P}$.

Ejercicio 17: Probar que $\mathcal{L}$ es decidable por una familia de circuitos L-uniformes $\iff \mathcal{L} \in \text{P}$.

Demostración:

$\subseteq$) Es consecuencia del hecho que $\text{L} \subseteq \text{P}$ y el **teorema 31**.

$\supseteq$) Viendo la demo del **teorema 27** de que $\text{P} \subseteq \text{P}_{\text{poly}}$, podemos ver que el tamaño del C_n es polinomial, pero podemos computar su codificación a partir de 1^n implícitamente en L. (el por qué te lo debo)

Proposición 43: $\text{PARITY} \in \text{NC}^1$.

Proposición 44: $\text{SUMA} \in \text{AC}^0$.

Proposición 45: $\text{NC}^d \subseteq \text{AC}^d \subseteq \text{NC}^{d+1}$. Por lo tanto, $\text{AC} = \text{NC}$.

Proposición 46: $\text{NC} \subseteq \text{P}$.

Teorema 32: $\mathcal{L}$ tiene una solución paralela eficiente sii $\mathcal{L} \in \text{NC}$.

Teorema 33: $\text{NC}^1 \subseteq \text{L}$.

Teorema 34: Sea $\mathcal{L} \in \text{NSpace}(S(n))$. Existe una familia de circuitos $(C_n)_{n \in \mathbb{N}}$ con compuertas $\wedge$ y $\vee$ de fan-in arbitrario y una máquina determinística M tal que para todo $n \geq 1$ y $x \in \{0, 1\}^n$ tenemos que $x \in \mathcal{L}$ sii $C_n(x) = 1$, $M(1^n) = \langle C_n \rangle$, $|C_n| = 2^{O(S(n))}$ y $\text{prof}(C_n) = O(S(n))$.

Corolario 8: $\text{NL} \subseteq \text{AC}^1$.

Proposición 47: CIRC-EVAL es P-completo.

Demostración:

El hecho de que $\text{CIRC-EVAL} \in \text{P}$ es bastante trivial porque sería reemplazar la entrada del circuito y recorrerlo con los valores.

Ahora el que $\text{CIRC-EVAL} \in \text{P-hard}$ lo vemos por el **ejercicio 17** ya que sabemos que para toda $\mathcal{L}' \in \text{P}$, $\mathcal{L}'$ es decidable por una familia de circuitos L-uniforme.

Con esto en cuenta solo nos queda hacer que $\mathcal{L}' \leq_{\ell} \text{CIRC-EVAL}$, para esto uso la f tal que $f(1^n) = \langle C_n \rangle$ que sabemos que es computable implícitamente por lo dicho antes.

Entonces queda que:

$$x \in \mathcal{L}' \iff C_{|x|}(x) \iff \langle f(1^{|x|}), x \rangle \in \text{CIRC-EVAL}$$

Teorema 35: Supongamos que $\mathcal{L}$ es P-completo:

1. $\mathcal{L} \in \text{NC} \iff \text{P} = \text{NC}$.

2. $\mathcal{L} \in \text{L} \iff \text{P} = \text{L}$.